

Quranic Chatbot

Code Guide

How the app's code works, explained step by step

Flutter app + FastAPI backend | Version 1.0.0

Table of Contents

1. The code in one minute.....	3
2. Architecture: five layers.....	3
3. How the app starts.....	4
4. Connecting everything together.....	5
5. State management.....	7
6. Data models.....	8
7. Services: talking to the backend safely.....	8
8. Feature walk-through: AI chat.....	10
9. Feature walk-through: Recite & Recognize.....	11
10. Recitation audio.....	12
11. Auto Play and the highlighted card.....	14
12. Local storage.....	15
13. Navigation.....	16
14. Language and themes.....	16
15. The backend in detail.....	16
16. Glossary.....	17
17. Where do I look when.....	17

How to use this guide. It explains what each part of the code does and why, with diagrams and annotated code pictures. You do not need to read it in order: every section starts with a one-line summary, and section 17 lists “where do I look when I want to change X”. File names refer to the project zip (`flutter_app/lib/...` and `backend/api/index.py`).

1. The code in one minute

The project has two programs that talk to each other:

Program	Language	Lives in	Job
The app	Dart (Flutter)	<code>flutter_app/lib/</code>	Everything the user sees and touches on the phone. Stores bookmarks, history and settings on the phone.
The backend	Python (FastAPI)	<code>backend/api/index.py</code>	Holds the secret API keys, asks Groq / Gemini for chat answers, transcribes speech, and matches a recitation to a real verse.

The app never talks to Groq, Gemini or the speech service directly. It only talks to the backend (and downloads verse audio from everyayah.com). That is why no secret key is inside the APK.

How the Dart code is split into files. `main.dart` lists every other file with a `part` line, and each of those files starts with `part of '../main.dart';`. Together they form **one Dart library**, so they share the same imports and can use each other's classes without `import` lines. Files whose names start with an underscore (for example `_pickKey`) are private to that library, not to a file. Splitting the code into folders changes how it is organised, not how it behaves.

2. Architecture: five layers

The code is arranged in layers. Each layer only uses the layer below it, which keeps changes local: you can replace how audio is played without touching any screen.

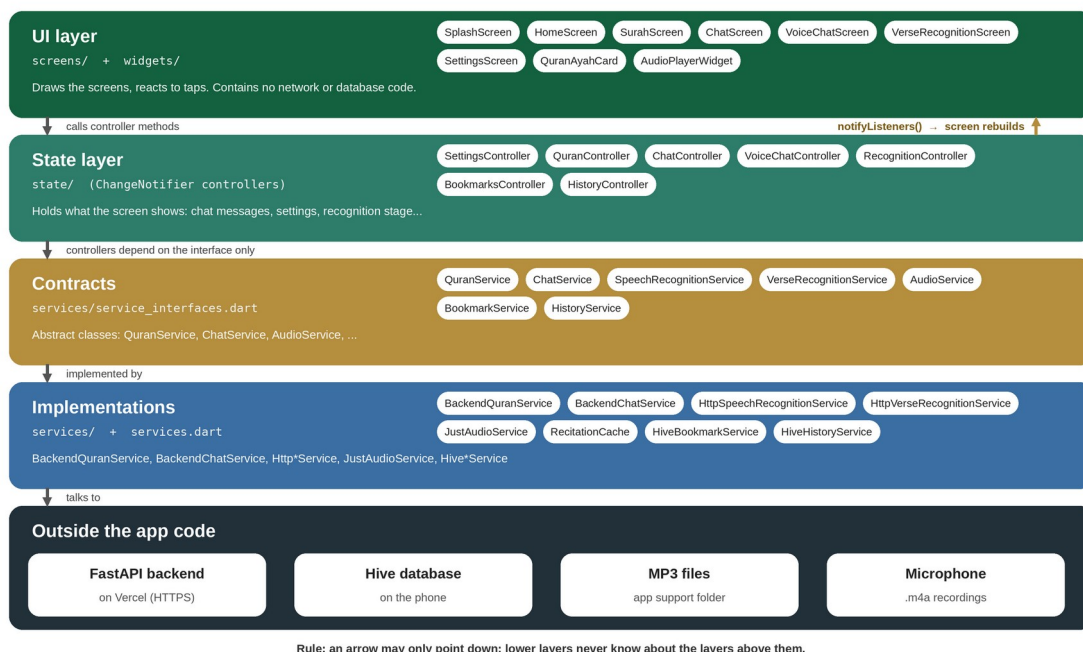


Figure 1: The five layers of the app, from what the user sees down to the outside world

Layer	Folder	Rule of thumb
UI	<code>screens/</code> , <code>widgets/</code>	Draws things and reacts to taps. Never calls the network or the database itself.
State	<code>state/</code>	Controllers (<code>ChangeNotifier</code>) hold what a screen shows and expose methods such as <code>send()</code> or <code>setLanguage()</code> .
Contracts	<code>services/</code> <code>service_interfaces.dart</code>	Abstract classes that describe <i>what</i> a service can do, not <i>how</i> .
Implementations	<code>services/</code> , <code>services.dart</code>	The real code: HTTP calls, Hive storage, audio player, file cache.
Outside	backend, Hive, files, microphone	Things the app talks to.

Supporting folders: `models/` (data classes), `data/` (built-in Surah names and sample verses), `utils/` (helper functions), `l10n/` (all text in three languages), `theme/` (colours and the 12 themes), `app/` (app root and routes).

3. How the app starts

Summary: `main()` prepares the local database, unpacks the offline Quran once, then starts the UI. The first screen decides where to go next.

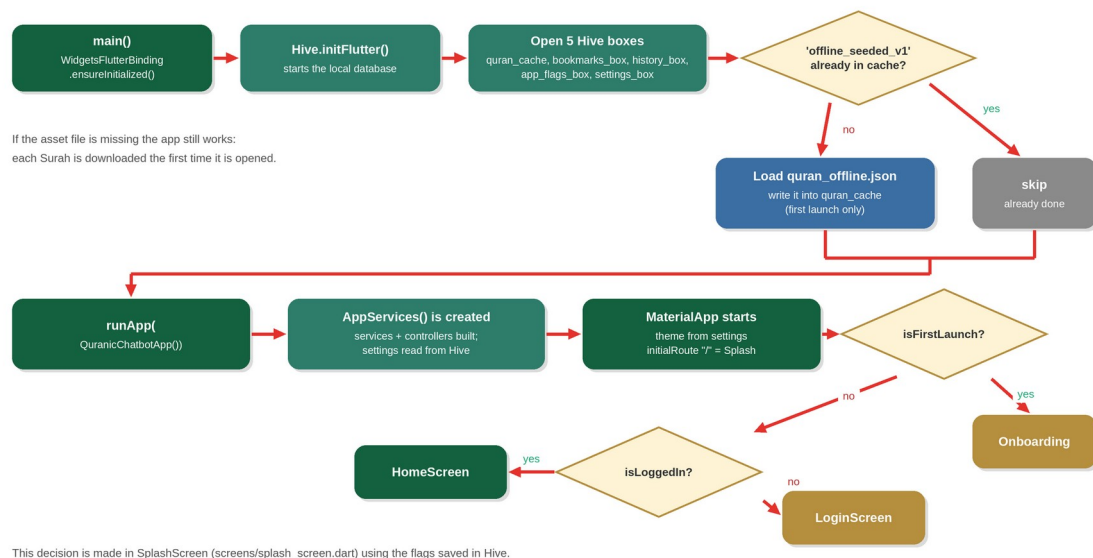


Figure 2: Start-up sequence, from `main()` to the first screen

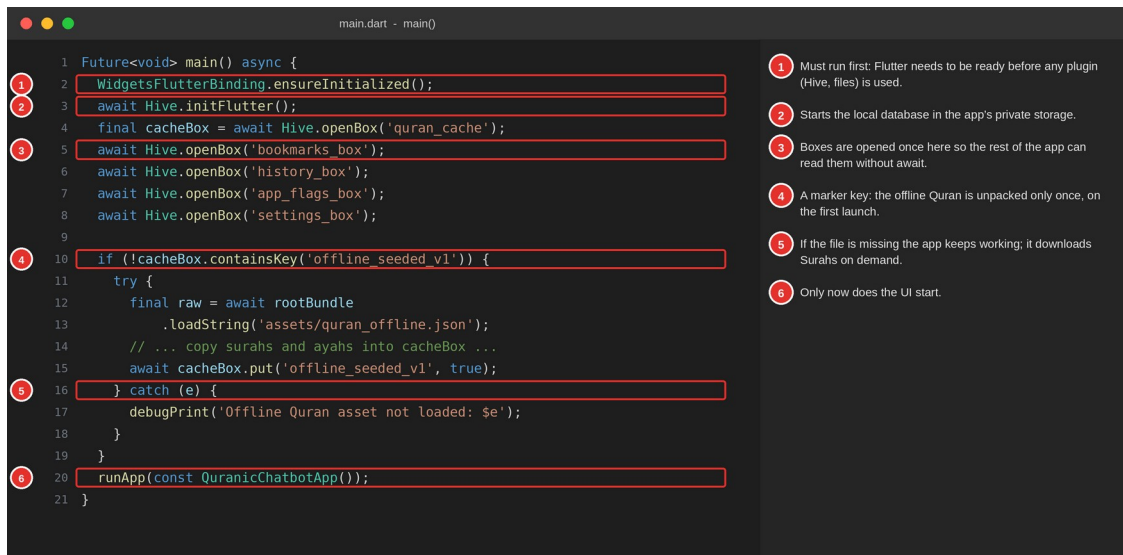


Figure 3: main() in main.dart, annotated

Points worth knowing:

- **WidgetsFlutterBinding.ensureInitialized()** is required whenever **main()** is **async** and uses plugins before **runApp**.
- **Five Hive boxes** are opened up front (**await Hive.openBox(...)**). After that, any code can call **Hive.box('name')** synchronously. That is why controllers can read settings in their constructors.
- **Offline seeding.** If **assets/quran_offline.json** exists, the whole Quran is copied into the **quran_cache** box once. The key **offline_seeded_v1** marks that it has been done. If the asset is missing, the **catch** prints a message and the app carries on; Surahs are then downloaded on first open and cached.
- **Where the user lands.** **SplashScreen** reads two flags saved in Hive: **isFirstLaunch** (go to Onboarding) and **isLoggedIn** (Home if true, Login if false).

4. Connecting everything together

Summary: **AppServices** creates every service and controller once. **AppScope** hands them to any widget through **context**.

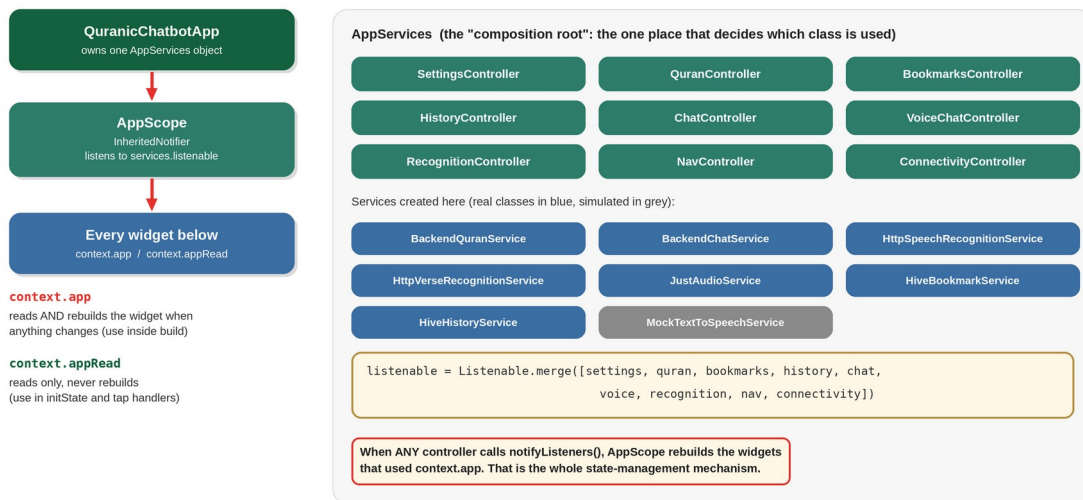


Figure 4: AppServices (composition root) and AppScope

AppServices (in `app/app.dart`) is called the *composition root*: it is the one place that decides which class is used for each job. For example:

```
late final ChatService chatService =
  BackendChatService(httpChat, quranService);
final AudioService audio = JustAudioService();
final BookmarkService bookmarkService = HiveBookmarkService();
```

To switch a feature back to fake data for UI experiments you change one line here (for example use `MockAudioService()`), and nothing else in the app changes. The mock classes in `services/mock_services.dart` exist for that purpose. Text-to-speech is still a mock (`MockTextToSpeechService`), so voice replies are shown as text.

Reading services from a widget. **AppScope** is an `InheritedNotifier`, and `app/app.dart` adds two shortcuts:

Shortcut	Behaviour	Use it in
<code>context.app</code>	Reads and rebuilds the widget whenever any controller changes	<code>build()</code> methods
<code>context.appRead</code>	Reads only, never rebuilds	<code>initState</code> , button handlers

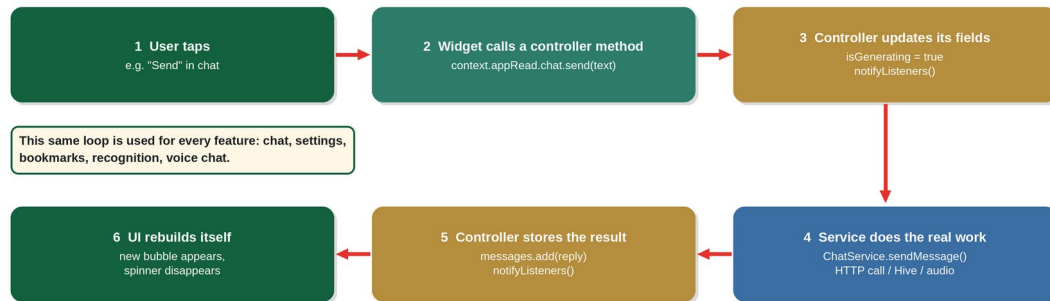
`AppServices.listenable` merges all controllers with `Listenable.merge([...])`. When any of them calls `notifyListeners()`, **AppScope** notifies the widgets that used `context.app`.

The root widget (`QuranicChatbotApp`) also wires three app-wide things:

- **Theme.** An `AnimatedBuilder` watches `settings`. If the theme is `system` it uses Classic light and dark and follows the phone; otherwise it pins the chosen theme.
- **Right-to-left.** It wraps the app in `Directionality` using `settings.isRtl`, so Urdu and Arabic mirror the layout without reversing any strings.
- **Audio stop on back.** `AudioStopObserver` is a `NavigatorObserver` that stops recitation when a full page is closed (dialogs and bottom sheets do not count).

5. State management

Summary: widgets never change data. They call a controller method; the controller changes its fields and calls `notifyListeners()`; the screen rebuilds.



Widgets never change data themselves; they only call controller methods and rebuild when told.

Figure 5: The loop used by every feature

The controllers in `state/`:

Controller	Holds	Main methods
<code>SettingsController</code>	Theme, language, font size, translation, reciter, speed, <code>autoPlay</code> , last read position, first-launch/login flags	<code>setThemeId</code> , <code>setLanguage</code> , <code>setAutoPlay</code> , ... (each saves to Hive and notifies)
<code>QuranController</code>	Surah list and load state (<code>loading</code> / <code>loaded</code> / <code>empty</code> / <code>error</code>); caches verses per Surah and Juz in memory	<code>loadSurahs</code> , <code>getAyahs</code> , <code>getJuz</code>
<code>ChatController</code>	Current conversation, <code>isGenerating</code> , <code>hasError</code>	<code>send</code> , <code>retry</code> , <code>newConversation</code> , <code>startWithAyah</code>
<code>VoiceChatController</code>	Voice state (<code>idle</code> , <code>listening</code> , <code>processing</code> , <code>speaking</code> , <code>error</code>), timer, transcript, reply	<code>start</code> , <code>send</code> , <code>cancel</code>
<code>RecognitionController</code>	Recognition stage, timer, result or error	<code>start</code> , <code>stop</code> , <code>reset</code>
<code>BookmarksController</code> , <code>HistoryController</code>	Saved verses and past chats	<code>load</code> , <code>add/remove/save/delete</code>
<code>NavController</code> , <code>ConnectivityController</code>	Selected bottom tab; offline banner flag	<code>go</code> , <code>setOffline</code>

A setting works like this: `SettingsController` reads every value from Hive in its constructor (with defaults, for example `autoPlay` defaults to `false`), and each setter does two things:

```
void setAutoPlay(bool v) {  
  _box.put('autoPlay', v);           // 1. save on the phone  
  _u(() => autoPlay = v);           // 2. change the field and notifyListeners()  
}
```

6. Data models

Summary: small immutable classes in `models/`. The important one is `Ayah`.

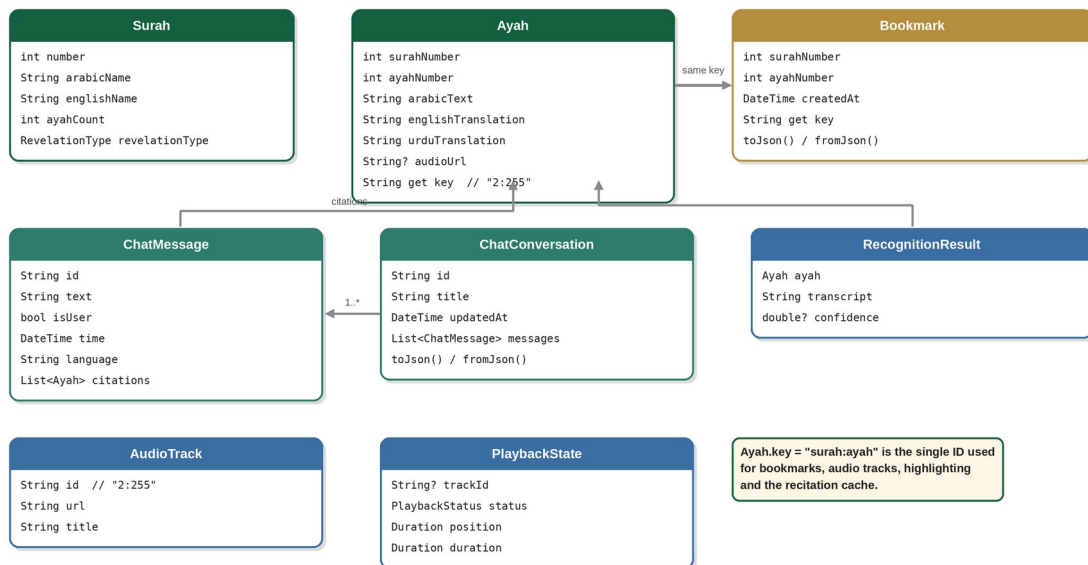


Figure 6: The main data classes

- **Ayah.key** is "surah:ayah" (for example "2:255"). The same string identifies a verse for bookmarks, audio tracks, card highlighting and the recitation cache.
- `Ayah`, `Bookmark`, `ChatMessage` and `ChatConversation` have `toJson()` / `fromJson()` so they can be stored in Hive as JSON text.
- `Ayah.translation('ur')` returns the Urdu text, anything else returns English.
- `RecognitionResult.confidence` is nullable (the backend sends it, the mock does not).

7. Services: talking to the backend safely

Summary: a service interface says what can be done; the `Backend*` and `Http*` classes do it by calling the backend, and cope with imperfect data.

Interface	Real implementation	What it does
<code>QuranService</code>	<code>BackendQuranService</code>	Surahs, verses, Juz, search, featured verse. Falls back to built-in sample data for the Surah list and featured verse.
<code>ChatService</code>	<code>BackendChatService</code>	Sends the question, turns the reply into a <code>ChatMessage</code> with real <code>Ayah</code> citations.
<code>SpeechRecognitionService</code>	<code>HttpSpeechRecognitionService</code>	Records a <code>.m4a</code> file and sends it to <code>/stt</code> .
<code>VerseRecognitionService</code>	<code>HttpVerseRecognitionService</code>	Records, sends to <code>/recognize</code> , converts errors to friendly types.
<code>AudioService</code>	<code>JustAudioService</code>	Plays recitation (section 11).
<code>BookmarkService</code> , <code>HistoryService</code>	<code>HiveBookmarkService</code> , <code>HiveHistoryService</code>	Save JSON in Hive.

Interface	Real implementation	What it does
TextToSpeechService	MockTextToSpeechService	Simulated.

The HTTP layer (**lib/services.dart**). It holds the constant `kBackendBaseUrl` and two classes:

- **HttpQuranService** fetches `/surahs`, `/surahs/{id}/ayahs`, `/juz/{id}` and `/search`. Quran text never changes, so responses are stored in the `quran_cache` box and reused. This is the same cache the offline Quran file fills.
- **HttpChatService** sends `/chat`, `/stt` and `/recognize` (audio is sent as a multipart file). Any non-2xx response throws an error whose text contains the status code and body. The rest of the app recognises words such as `404` or `no_match` in that text to show a friendly message.

Note. The `services.dart` in the project zip was written to match exactly how the rest of the app calls it (same class and method names, same return types). If you replace it with your own version, keep those names and types and everything described here still works.

Tolerant parsing. `_parseAyah` accepts several possible field names for each value, so a small change in the backend's JSON does not break the screens.

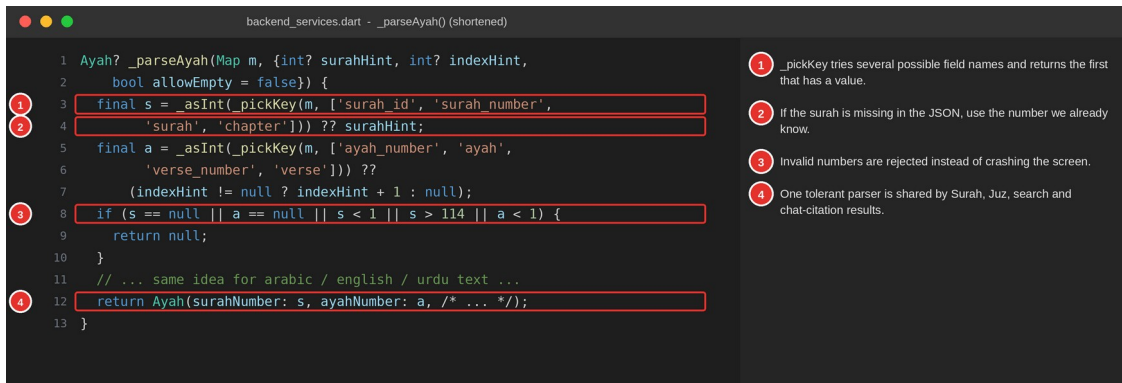


Figure 7: `_parseAyah` tries several field names and rejects invalid numbers

Chat citations. The backend returns citations as strings such as `"2:153"`. **BackendChatService** finds the numbers with the pattern `(\d{1,3})\s*[:\-\]\s*(\d{1,3})`, then loads the real verse text with `QuranService.getAyahs`, so the citation cards show verified Quran text, not text written by the AI. Duplicate citations are removed.

8. Feature walk-through: AI chat

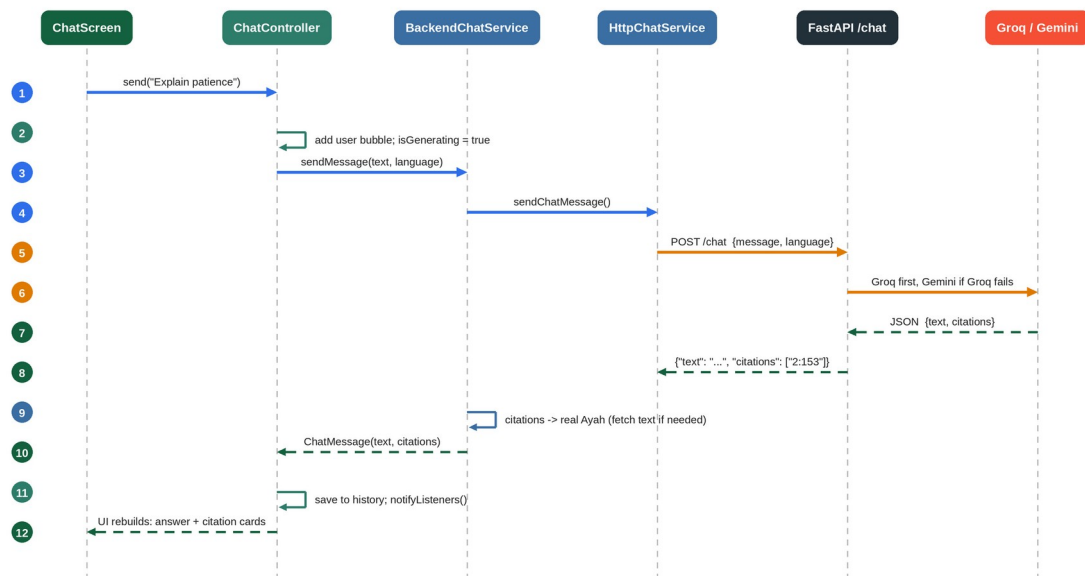


Figure 8: One chat message from tap to answer

1. `ChatController.send()` ignores empty text and ignores taps while a reply is already being generated (`isGenerating`).
2. It creates the conversation on first use (title = first 40 characters), adds the user bubble, sets `isGenerating = true` and notifies.
3. `BackendChatService` calls the backend with the *response language* setting, not the interface language.
4. On success the reply is added and the conversation is saved to Hive (`HistoryController.save`).
5. On failure `hasError = true`; the screen shows an error bar with **Retry**, which calls `retry()` with the last user message.

`startWithAyah` starts a new chat with the text “Explain Surah X, Ayah N”, which is what the “ask about this verse” action uses.

Voice chat reuses the same path. `VoiceChatController` records audio (`start`), and on `send` uploads it to `/stt`, then passes the returned text to `ChatController.send`. Its states are `idle` → `listening` → `processing` → `idle` (or `error`). There is no live partial transcript because speech-to-text is request/response, not streaming.

9. Feature walk-through: Recite & Recognize

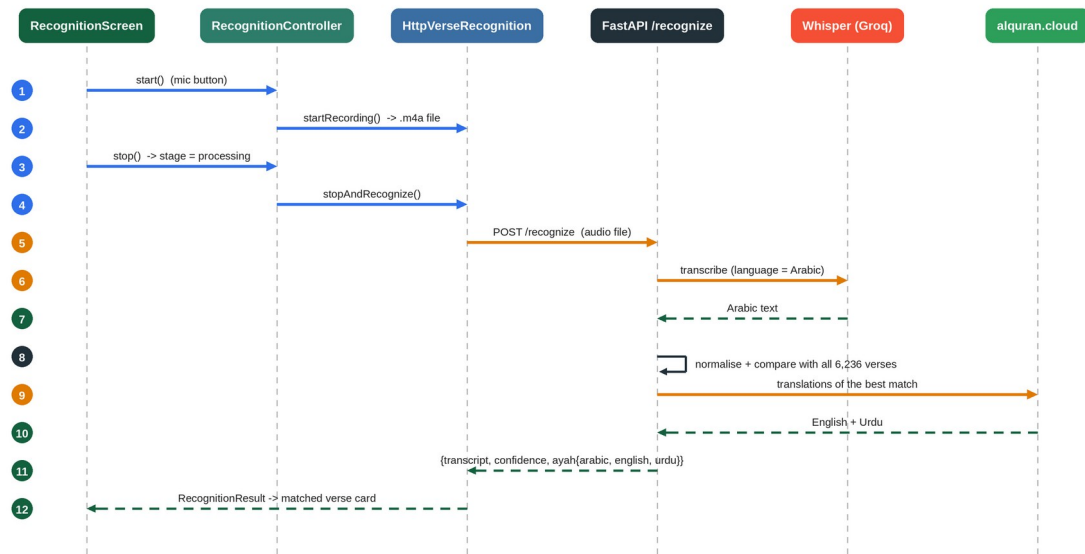


Figure 9: From microphone to matched verse

RecognitionController is a small state machine with these stages: **initial** → **recording** → **processing** → **result** or **failure**. Recording is limited to **30 seconds**; a timer stops it automatically.

HttpVerseRecognitionService turns failures into typed errors so the screen can show the right message:

Situation	Error type shown
Microphone permission denied	microphonePermission
Network failure (socket, timeout, host lookup)	network
Backend says 400 / no_speech	noSpeech
Backend says 404 / no_match	noMatch
Anything else	serviceUnavailable

The network check comes first on purpose: a dropped connection can produce an error text that happens to contain digits, and it must not be mistaken for a status code.

On success the service builds an **Ayah** from the backend's answer. If the Arabic, English or Urdu text is missing, it looks the verse up itself with **QuranService.getAyahs**, so all three languages always appear.

On the backend the matching is plain text comparison, not AI:

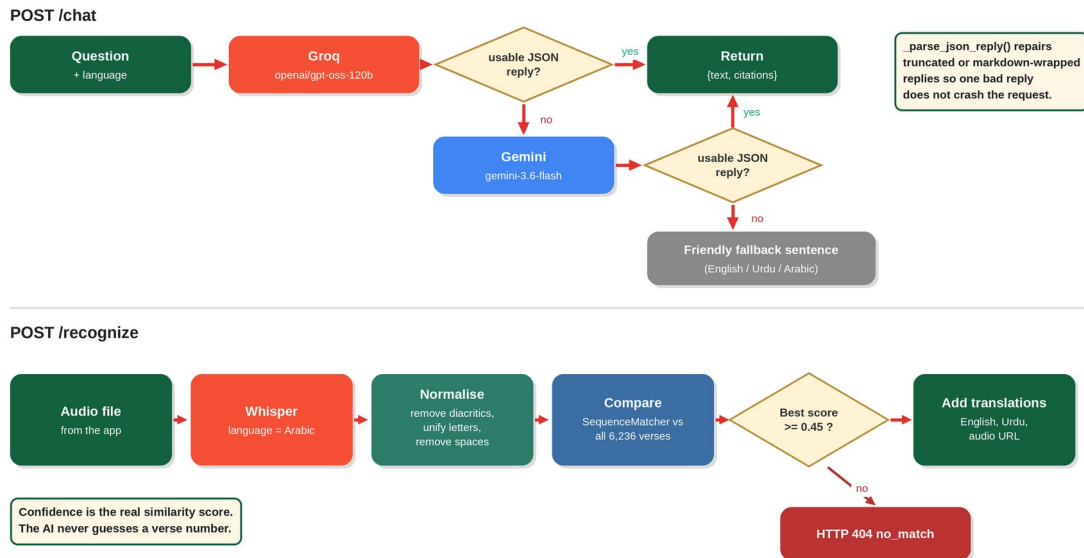


Figure 10: The two backend flows: /chat and /recognize

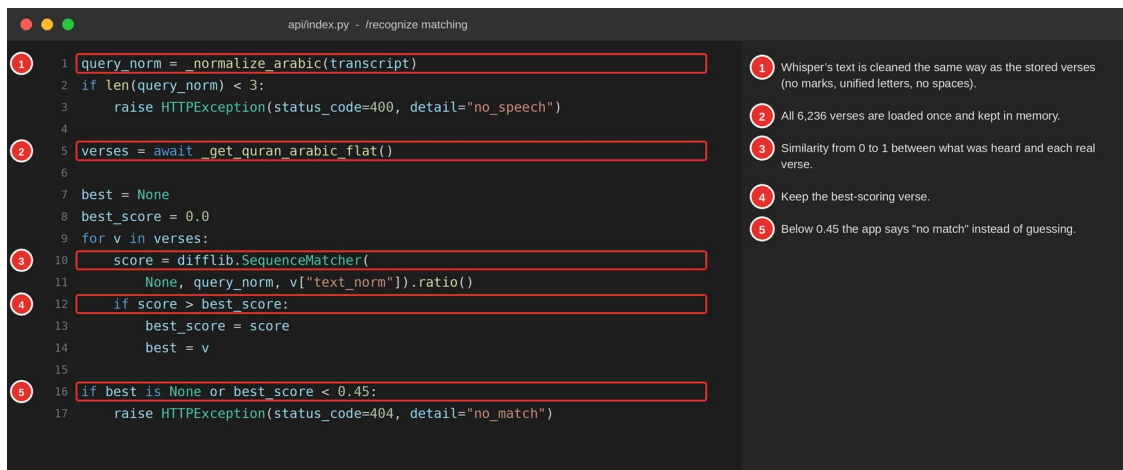
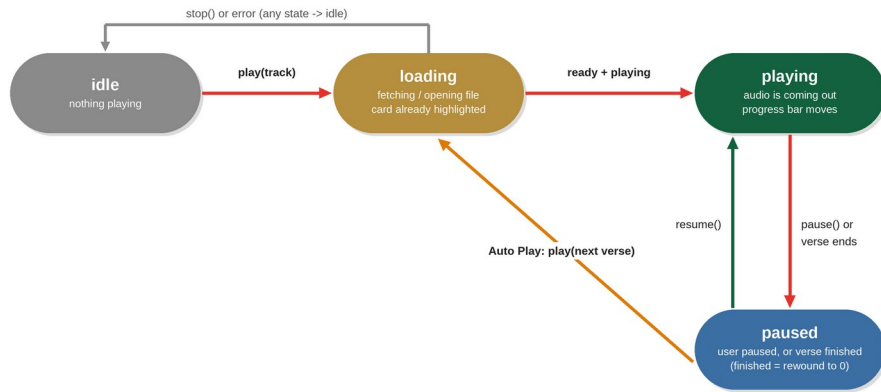


Figure 11: The matching loop in api/index.py

`_normalize_arabic` removes diacritics, unifies letter variants (for example the different forms of alef) and removes spaces, because the speech model's spelling and spacing rarely match the Uthmani script exactly. The same function is applied to all 6,236 stored verses once, so both sides are comparable. The returned confidence is the real similarity score.

10. Recitation audio

Summary: `JustAudioService` maps the `just_audio` player onto the app's own `PlaybackState`, so the UI does not depend on the player package.



Three flags stop the state from flickering (JustAudioService)

_switching
true while a new verse is being opened; player events of the OLD verse are ignored

_starting
play() was issued but the player is not reporting "playing" yet; keeps "loading" (not "paused")

_completionFired
a verse reports "finished" exactly ONCE, so Auto Play never skips a verse

Figure 12: Playback states

PlaybackState holds **trackId** (the verse key), **status** (**idle** / **loading** / **playing** / **paused**), **position** and **duration**. Widgets listen to **audio.state** (a **ValueListenable**) to draw the play/pause button, the progress bar and the highlighted card.

The trickiest part is **_onPlayerState**. The player emits many events, some of them belonging to the *previous* verse. Three flags protect the UI:

```

1 void _onPlayerState(ja.PlayerState ps) {
2   if ( trackId == null || _switching) return;
3   switch (ps.processingState) {
4     case ja.ProcessingState.ready:
5       if (ps.playing) {
6         _starting = false;
7         _state.value = _state.value
8           .copyWith(status: PlaybackStatus.playing);
9       }
10      break;
11     case ja.ProcessingState.completed:
12       if (_completionFired) return;
13       _completionFired = true;
14       final finished = _trackId!;
15       _player.pause();
16       _player.seek(Duration.zero);
17       _state.value = _state.value.copyWith(
18         status: PlaybackStatus.paused,
19         position: Duration.zero);
20       _completeCtrl.add(finished);
21       break;
22   }
23 }

```

- 1 Events that belong to the PREVIOUS verse are ignored while a new one is loading.
- 2 Only when the player really reports "playing" does the UI switch to playing.
- 3 just_audio can report "completed" twice. Without this guard Auto Play would skip a verse.
- 4 The finished verse rewinds to 0:00 and waits, so the play button replays it.
- 5 Announces "verse finished" to whoever listens (the Surah / Juz screens).

Figure 13: Handling player events safely

play(track) also decides where the sound comes from:

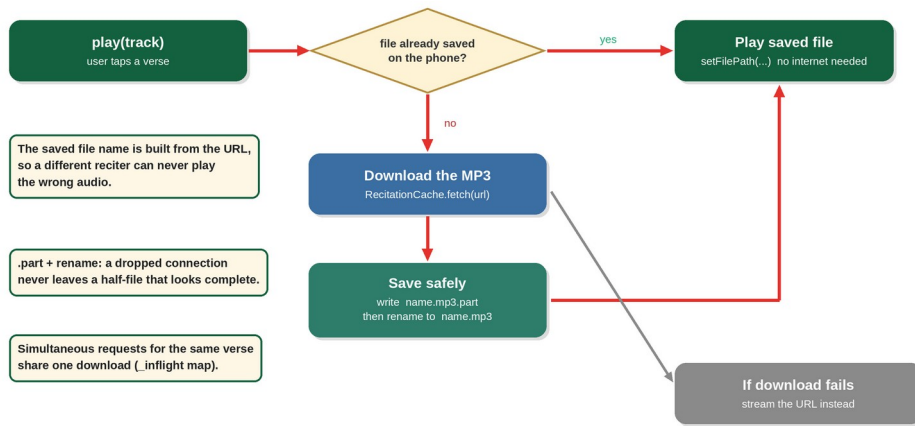


Figure 14: Play from a saved file, otherwise download once

`RecitationCache` stores files in the app's private support folder under `recitations/`. The file name is built from the last two parts of the URL, so different reciters or bitrates never collide. Settings reads `stats()` to show how many files are saved and their total size.

11. Auto Play and the highlighted card

Summary: when a verse ends, the reader screen starts the next one, lights its card and scrolls to it.

`JustAudioService` announces the end of a verse on `onTrackComplete`. `SurahScreen` and `JuzScreen` each listen to that stream and run `_onTrackComplete`:

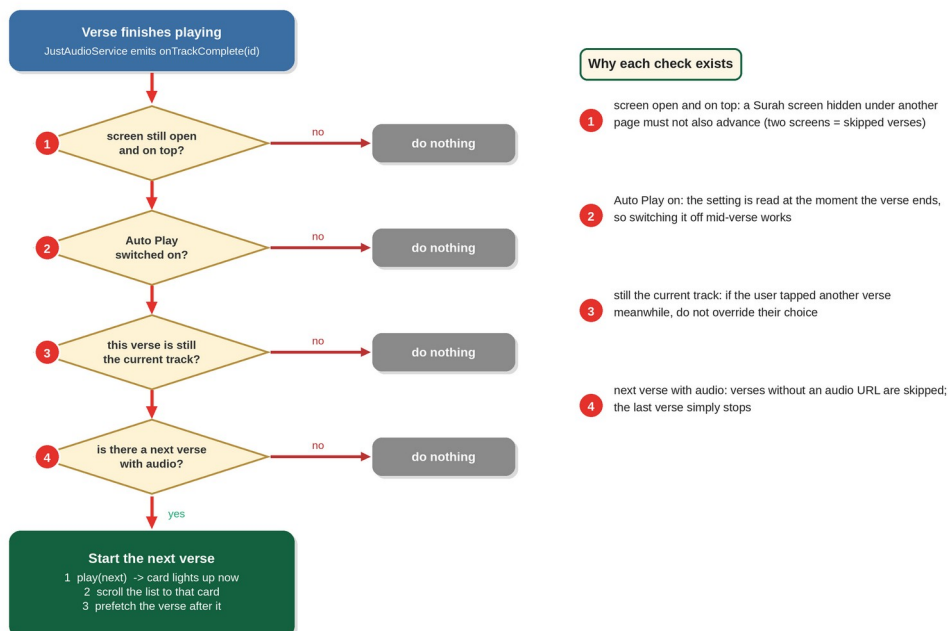


Figure 15: The four checks before the next verse starts

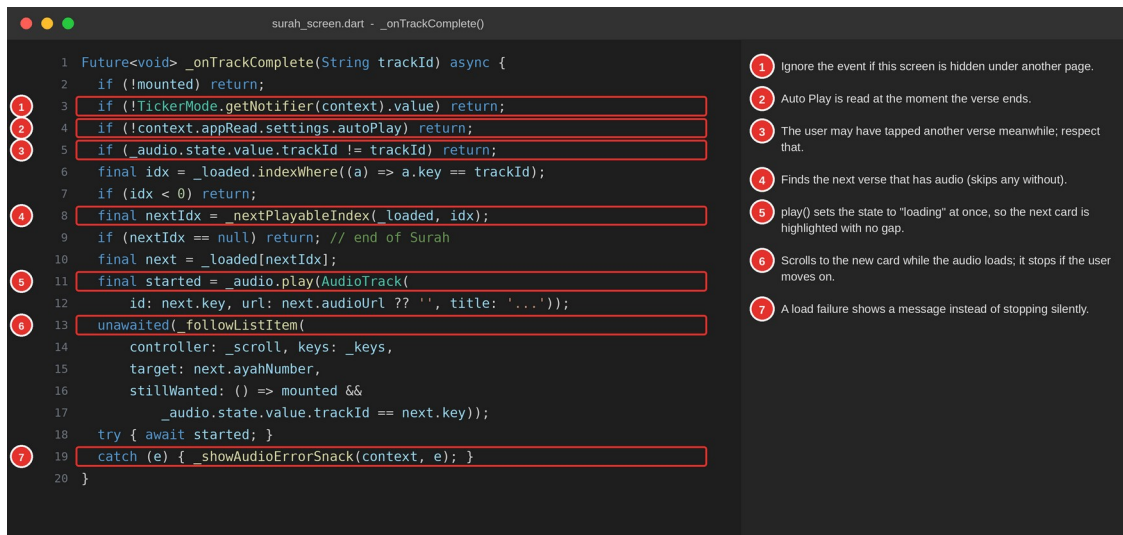


Figure 16: `_onTrackComplete` in `surah_screen.dart`

Supporting pieces:

- **`_nextPlayableIndex`** returns the next verse that has an audio URL (verses without one are skipped).
- **Prefetching.** While a verse plays and Auto Play is on, `_onAudioState` downloads the *following* verse in the background (`_prefetchRecitation`), so the hand-off has no waiting.
- **`_followListItem`** scrolls to the new card. A `ListView.builder` only builds cards near the screen, so the target card may not exist yet; the helper scrolls a screenful at a time towards it until it appears, then eases it into view. It stops if the user starts another verse (`stillWanted`).
- **Highlight.** `QuranAyahCard` compares the card's `ayah.key` with `audio.state.trackId` and highlights it while the status is `playing`, `loading`, or `paused` part-way through. `AppCard` draws the gold tint, thick border and glow when `highlight` is true.
- **Speed.** The chosen playback speed is applied to each new verse.

12. Local storage

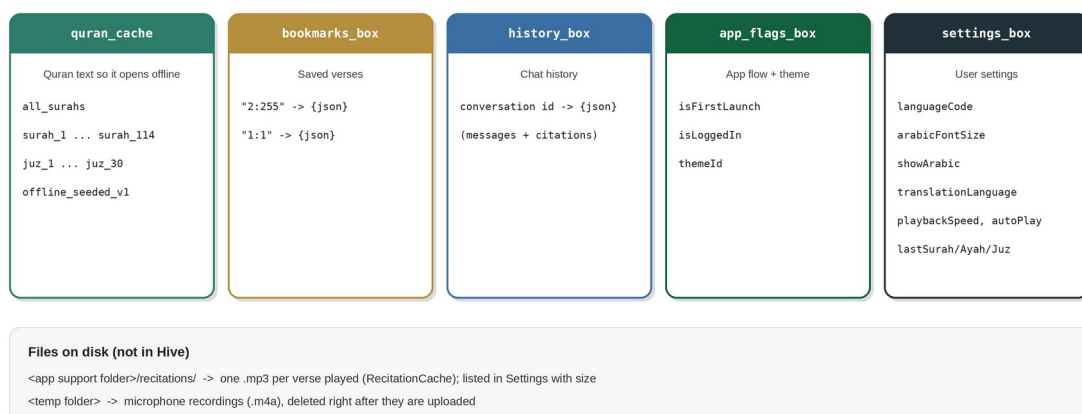


Figure 17: What is stored on the phone

Hive stores plain values by key. Bigger objects are saved as JSON text (`jsonEncode(object.toJson())`) and read back with `fromJson`. Nothing is sent to a server: bookmarks, history, settings and login flags exist only on the device. Uninstalling the app deletes them.

The Login and Signup screens are **local accounts** that decide the app flow only; they are not real authentication (the backend has no login endpoints).

13. Navigation

`app/routes.dart` defines the route names in `AppRoutes` (for example `/surah`, `/verse-recognition`) and a single `onGenerateRoute` that maps each name to a screen. Routes with data use small argument classes: `SurahArgs(surahNumber, [initialAyah])` and `JuzArgs(juzNumber)`. `AppPageRoute` gives every page the same fade-and-slide transition; it slides from the left in right-to-left languages.

The bottom tabs are handled by `MainShell`, driven by `NavController.index`. `AppNav.goToTab` changes the tab and pops back to the first route. `PermissionFlow.ensureMicrophone` shows an explanation dialog before asking for the microphone permission.

14. Language and themes

- **Text.** Every visible string lives in `l10n/strings.dart` in three maps (`en`, `ur`, `ar`). `context.tr('key')` looks up the current language and falls back to English, then to the key itself. To add text, add the same key to all three maps.
- **Direction.** `settings.isRtl` is true for every language except English; the root `Directionality` mirrors the layout.
- **Themes.** `AppThemeCatalog` defines 12 themes (Classic light and dark, Blossom, Serika Dark, Dracula, Nord, Gruvbox, Catppuccin Mocha, Solarized Dark, Monokai, Tokyo Night, Rosé Pine) as colour specs. `AppTheme.build(spec)` turns a spec into Flutter's `ThemeData`. Widgets use `context.accentGold`, `context.cardColor` and similar shortcuts instead of fixed colours, which is why a theme change restyles every screen.

15. The backend in detail

File: `backend/api/index.py` (run by Vercel through `vercel.json`).

Part	What it does
Start-up	Creates the FastAPI app with permissive CORS; reads <code>GROQ_API_KEY</code> and <code>GEMINI_API_KEY</code> from environment variables and stops with an error if either is missing
Caches	Python dictionaries keep the Surah list, each Surah, each Juz and the whole Quran text in memory while the function is warm
<code>/surahs</code> , <code>/surahs/{id}/ayahs</code> , <code>/juz/{id}</code> , <code>/search</code>	Fetch from alquran.cloud (Arabic <code>quran-uthmani</code> , English <code>en.sahih</code> , Urdu <code>ur.jalandhry</code>) and return simple JSON; each verse includes an everyayah.com <code>audio_url</code>
<code>/chat</code>	Groq first, then Gemini, then a fixed fallback sentence; <code>_parse_json_reply</code> repairs damaged JSON
<code>/stt</code>	Whisper transcription of an uploaded file
<code>/recognize</code>	Section 9: transcribe, normalise, compare, attach translations

The chat prompt tells the model to answer in the requested language with 2 to 3 sentences and to return only JSON of the form `{"text": "...", "citations": ["surah:ayah"]}`. The app then verifies each citation against the real Quran text (section 7).

Because Vercel runs the backend as a serverless function, the memory caches survive only while the function stays warm, which is why the first request after a quiet period is slower.

16. Glossary

Term	Meaning in this project
Widget	A piece of UI in Flutter (a button, a card, a whole screen).
StatefulWidget	A widget that can change while it is on screen (for example the Surah reader).
ChangeNotifier	A Flutter class that can tell listeners “I changed” with <code>notifyListeners()</code> . All controllers extend it.
InheritedNotifier	A widget that makes an object available to everything below it (used by <code>AppScope</code>).
Future / async / await	A value that arrives later (a network reply). <code>await</code> waits for it without freezing the screen.
Stream	A series of values over time (for example “verse finished” events).
Hive	The small on-device database. A <i>box</i> is like a named dictionary.
Interface (abstract class)	A list of methods that any implementation must provide.
part / part of	Dart’s way of spreading one library over several files.
Composition root	The single place where objects are created and connected (<code>AppServices</code>).
Serverless	Code that runs only when a request arrives (Vercel), with no server to manage.
CORS	A browser rule about which websites may call an API; set to allow all here.

17. Where do I look when...

I want to...	Open
Change the backend address	<code>lib/services.dart</code>
Change or add text in any language	<code>lib/l10n/strings.dart</code>
Change colours or add a theme	<code>lib/theme/theme_catalog.dart</code>
Change how a verse card looks or is highlighted	<code>lib/widgets/verse_widgets.dart</code> and <code>AppCard</code> in <code>lib/widgets/ui_primitives.dart</code>
Change Auto Play	<code>lib/screens/surah_screen.dart</code> , <code>lib/screens/juz_screen.dart</code> , <code>lib/services/just_audio_service.dart</code>
Change how audio is saved offline	<code>lib/services/recitation_cache.dart</code>
Change a default setting	<code>lib/state/settings_controller.dart</code>
Switch a feature between real and mock data	<code>AppServices</code> in <code>lib/app/app.dart</code>

I want to...	Open
Add a screen	New file in <code>lib/screens/</code> , a <code>part</code> line in <code>main.dart</code> , a route in <code>lib/app/routes.dart</code>
Change the chat prompt or AI models	<code>SYSTEM_PROMPT</code> , <code>GROQ_CHAT_MODEL</code> , <code>GEMINI_CHAT_MODEL</code> in <code>backend/api/index.py</code>
Change how strict verse matching is	The <code>0.45</code> threshold in <code>/recognize</code>
Debug a network problem	The Backend Test screen (<code>/backend-test</code>) and the <code>[audio] / Backend ... failed</code> lines in the <code>flutter run</code> console